2520030117

SKILL–21: Module Integration, Error Handling and Refactoring

AIM

To connect different modules, validate interfaces, test end-to-end execution, handle runtime errors, improve program reliability, and refactor components for better code quality.

OBJECTIVE

To detect runtime errors, generate meaningful error messages, log failures, handle invalid input/syntax gracefully, recover from errors, and test different error scenarios.

PROCEDURE

1. Create separate modules/classes for different functionalities.
2. Define proper interfaces between the modules.
3. Validate the input before processing.
4. Connect all modules and execute the complete flow.
5. Use exception handling to detect runtime errors.
6. Generate meaningful error messages for invalid input.
7. Log failures for debugging.
8. Handle errors gracefully without terminating the program unexpectedly.
9. Test valid and invalid input scenarios.
10. Refactor repeated or unnecessary code to improve reliability and readability.

PROGRAM

```
import java.util.Scanner;

class InputValidator {

    public boolean validate(String name, int age) {
        if (name == null || name.trim().isEmpty()) {
            throw new IllegalArgumentException("Name cannot be empty.");
        }

        if (age < 0 || age > 120) {
            throw new IllegalArgumentException("Age must be between 0 and 120.");
        }

        return true;
    }
```

```
}

class UserService {

    public String processUser(String name, int age) {
        return "User " + name + " registered successfully with age " +
    age; }
}

class ErrorLogger {

    public void logError(String message) {
        System.out.println("[ERROR LOG] " + message);
    }
}

public class Skill21 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        InputValidator validator = new InputValidator();
        UserService service = new UserService();
        ErrorLogger logger = new ErrorLogger();

        try {
            System.out.print("Enter name: ");
            String name = sc.nextLine();

            System.out.print("Enter age: ");
            int age = sc.nextInt();

            // Validate input
            validator.validate(name, age);

            // End-to-end module execution
            String result = service.processUser(name, age);

            System.out.println("\n" + result);
            System.out.println("End-to-end execution completed successfully.");

        } catch (IllegalArgumentException e) {

            System.out.println("\nInvalid Input: " + e.getMessage());
            logger.logError(e.getMessage());
            System.out.println("Program recovered gracefully.");
```

```
        } catch (Exception e) {

            System.out.println("\nRuntime Error: Invalid input
            format."); logger.logError(e.getMessage());
            System.out.println("Program recovered gracefully.");

        } finally {

            sc.close();
            System.out.println("Program execution completed.");
        }
    }
}
```

TEST CASE 1 – VALID INPUT

Input:

Enter name: Kruthi
Enter age: 18

Output:

User Kruthi registered successfully with age 18
End-to-end execution completed successfully.
Program execution completed.

TEST CASE 2 – INVALID AGE

Input:

Enter name: Kruthi
Enter age: 150

Output:
Invalid Input: Age must be between 0 and 120.
[ERROR LOG] Age must be between 0 and 120.
Program recovered gracefully.
Program execution completed.

TEST CASE 3 – EMPTY NAME

Input:

Enter name:
Enter age: 18

Output:

Invalid Input: Name cannot be empty.
[ERROR LOG] Name cannot be empty.
Program recovered gracefully.
Program execution completed.

TEST CASE 4 – INVALID AGE FORMAT

Input:

Enter name: Kruthi
Enter age: abc

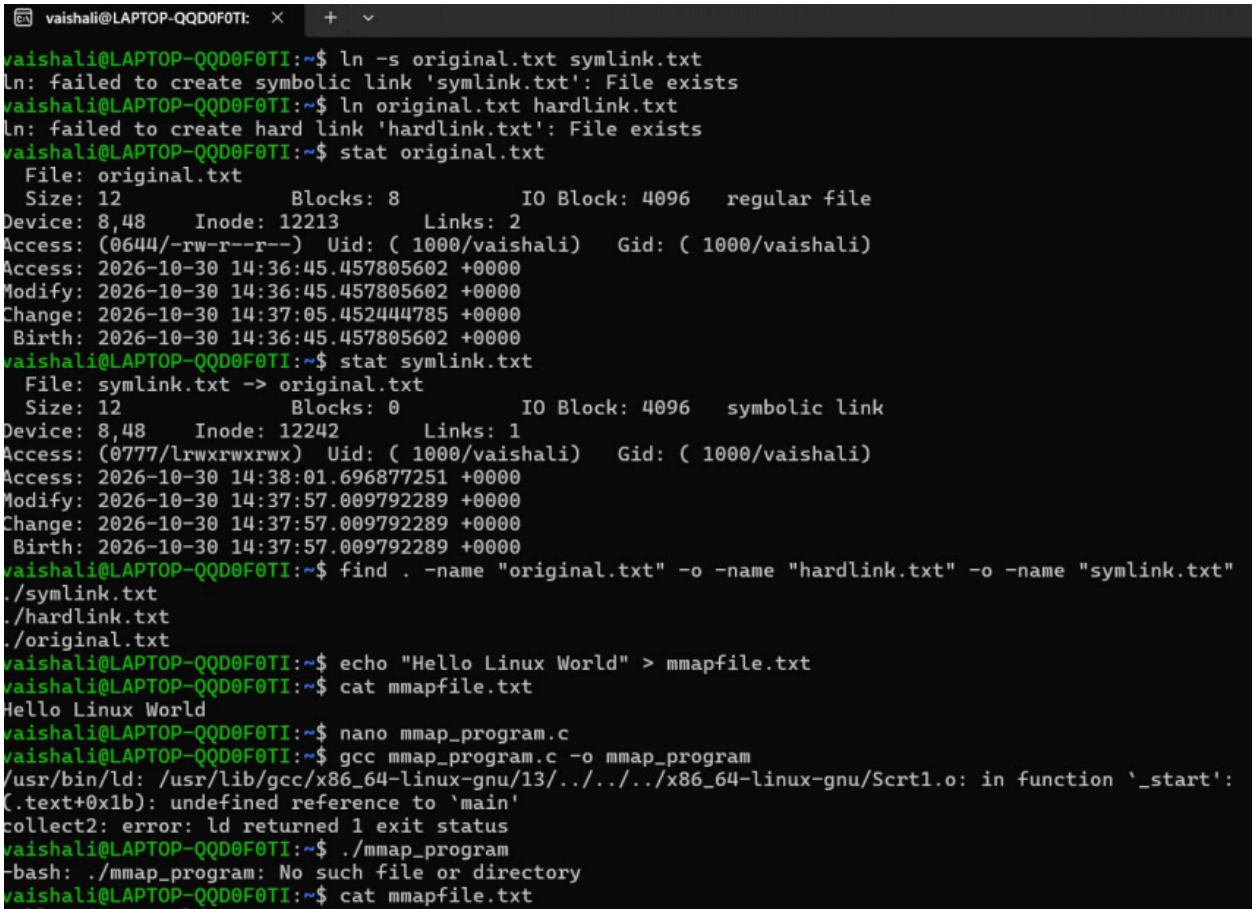
Output:

Runtime Error: Invalid input format.
[ERROR LOG] null
Program recovered gracefully.
Program execution completed.

RESULT

Thus, the modules were successfully connected and tested using end-to-end execution. Invalid inputs and runtime errors were detected and handled gracefully using exception handling and error logging, improving the reliability and maintainability of the program.

```
n/jps to provide /usr/bin/jps (jps) in auto mode
update-alternatives: using /usr/lib/jvm/java-21-openjdk-amd64/bi
n/jrunscript to provide /usr/bin/jrunscript (jrunscript) in auto
 mode
update-alternatives: using /usr/lib/jvm/java-21-openjdk-amd64/bi
n/jshell to provide /usr/bin/jshell (jshell) in auto mode
update-alternatives: using /usr/lib/jvm/java-21-openjdk-amd64/bi
n/jstack to provide /usr/bin/jstack (jstack) in auto mode
update-alternatives: using /usr/lib/jvm/java-21-openjdk-amd64/bi
n/jstat to provide /usr/bin/jstat (jstat) in auto mode
update-alternatives: using /usr/lib/jvm/java-21-openjdk-amd64/bi
n/jstatd to provide /usr/bin/jstatd (jstatd) in auto mode
update-alternatives: using /usr/lib/jvm/java-21-openjdk-amd64/bi
n/jwebserver to provide /usr/bin/jwebserver (jwebserver) in auto
 mode
update-alternatives: using /usr/lib/jvm/java-21-openjdk-amd64/bi
n/serialver to provide /usr/bin/serialver (serialver) in auto mo
de
update-alternatives: using /usr/lib/jvm/java-21-openjdk-amd64/bi
n/jhsdb to provide /usr/bin/jhsdb (jhsdb) in auto mode
Setting up default-jre (2:1.21-75+exp1) ...
Setting up openjdk-21-jdk:amd64 (21.0.12.1+1-1~24.04.4) ...
update-alternatives: using /usr/lib/jvm/java-21-openjdk-amd64/bi
n/jconsole to provide /usr/bin/jconsole (jconsole) in auto mode
Setting up default-jdk-headless (2:1.21-75+exp1) ...
Setting up default-jdk (2:1.21-75+exp1) ...
```


vaishali@LAPTOP-QQD0F0TI:~$ ln -s original.txt symlink.txt
ln: failed to create symbolic link 'symlink.txt': File exists
vaishali@LAPTOP-QQD0F0TI:~$ ln original.txt hardlink.txt
ln: failed to create hard link 'hardlink.txt': File exists
vaishali@LAPTOP-QQD0F0TI:~$ stat original.txt
File: original.txt
Size: 12 Blocks: 8 IO Block: 4096 regular file
Device: 8,48 Inode: 12213 Links: 2
Access: (0644/-rw-r--r--) Uid: (1000/vaishali) Gid: (1000/vaishali)
Access: 2026-10-30 14:36:45.457805602 +0000
Modify: 2026-10-30 14:36:45.457805602 +0000
Change: 2026-10-30 14:37:05.452444785 +0000
Birth: 2026-10-30 14:36:45.457805602 +0000
vaishali@LAPTOP-QQD0F0TI:~$ stat symlink.txt
File: symlink.txt -> original.txt
Size: 12 Blocks: 0 IO Block: 4096 symbolic link
Device: 8,48 Inode: 12242 Links: 1
Access: (0777/lrwxrwxrwx) Uid: (1000/vaishali) Gid: (1000/vaishali)
Access: 2026-10-30 14:38:01.696877251 +0000
Modify: 2026-10-30 14:37:57.009792289 +0000
Change: 2026-10-30 14:37:57.009792289 +0000
Birth: 2026-10-30 14:37:57.009792289 +0000
vaishali@LAPTOP-QQD0F0TI:~$ find . -name "original.txt" -o -name "hardlink.txt" -o -name "symlink.txt"
./symlink.txt
./hardlink.txt
./original.txt
vaishali@LAPTOP-QQD0F0TI:~$ echo "Hello Linux World" > mmapfile.txt
vaishali@LAPTOP-QQD0F0TI:~$ cat mmapfile.txt
Hello Linux World
vaishali@LAPTOP-QQD0F0TI:~$ nano mmap_program.c
vaishali@LAPTOP-QQD0F0TI:~$ gcc mmap_program.c -o mmap_program
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/13/../../../x86_64-linux-gnu/Scrt1.o: in function `_start':
(.text+0x1b): undefined reference to `main'
collect2: error: ld returned 1 exit status
vaishali@LAPTOP-QQD0F0TI:~$ ./mmap_program
-bash: ./mmap_program: No such file or directory
vaishali@LAPTOP-QQD0F0TI:~$ cat mmapfile.txt